

Projektopgave Programmering B

Lasse Svendsen og Sebastian Muff Kristensen

Klasse: h25hx2v

Fag: Programmering B

Underviser: Christian Bruhn

HTX, EUC-Syd Haderslev

Tegn: 18354

8 maj 2026

Indholdsfortegnelse

1	Indledning	1
2	Problemformulering	2
3	Teori og metode	3
4	Analyse	5
5	Test og teststrategi	9
6	Konklusion	11

1 Indledning

I dette projekt er der blevet arbejdet med at udvikle et computerspil i programmet Godot. Formålet med projektet er at anvende grundlæggende viden om programmering i praksis og få en bedre forståelse for, hvordan et spil kan udvikles fra en idé til et færdigt produkt. Der har samtidig været fokus på planlægning af projektet og strukturering af kode, så spillet fungerer som en helhed. Der er udviklet et 2D-platformspil med titlen "Escape the School". I spillet styrer spilleren en karakter, som skal bevæge sig gennem en bane ved at løbe og hoppe over forhindringer. Undervejs skal forskellige forhindringer undgås, og nøgler skal samles for at kunne åbne en dør og gennemføre banen. Spillet er designet til at være let at forstå, men stadig give en vis udfordring. Projektet er blevet afgrænset, så fokuset har været på at udvikle et spil, der er realistisk at gennemføre inden for tidsrammen og niveauet indtil videre i programmering. Der er derfor taget udgangspunkt i de mest centrale funktioner, som er nødvendige for, at spillet fungerer. Herefter har der været mulighed for at udvide spillet med flere elementer, hvis det har været muligt og der er overskydende tid. Denne fremgangsmåde har bidraget til en mere overskuelig arbejdsproces og gjort det nemmere at prioritere undervejs.

2 Problemformulering

I denne opgave undrer vi os over, hvordan man kan lave et rigtigt 2D-spil i Godot, der reagere når spilleren trykker på knapperne. Hvordan kan det lade sig gøre, at et spil kan få figurer til at hoppe, undgå ting og samle objekter op, og er det noget vi selv kan fremstille med de midler vi har lært i programmering? Bag ved det visuelle spil er der en masse kode, der styrer bevægelse, kollisioner og point, og vi vil gerne finde ud af, hvordan det virker. Vi er motiveret til at undersøge, hvordan man gør samt at prøve at lave vores eget spil, fordi vi gerne vil se, hvad vi kan lave med det, vi har lært om variabler, betingelser, løkker, funktioner og anvendelse af objekter eller simple klasser. I stedet for at lave små opgaver og delvise spil ud fra instruktionsvideoer, vil vi gerne lave et spil fra bunden med interaktion, som f.eks. at flytte en figur rundt, der kan samle genstande op og holde styr på point. Hvordan kan vi designe og fremstille et 2D-spil i Godot med fokus på spillerinteraktion og spilmekanikker, og hvordan påvirker vores valg af programmeringsmetoder spillets brugeroplevelse?

- Hvilke programmeringsprincipper og værktøjer i Godot er relevante for udviklingen af et 2D-spil?
- Hvordan implementeres centrale spilmekanikker (fx bevægelse, kollision og scoring) i Godot ved hjælp af GDScript?
- I hvilken grad fungerer spillet teknisk og oplevelsesmæssigt, og hvilke forbedringer kunne øge kvaliteten?

3 Teori og metode

I udviklingen af spillet er der taget udgangspunkt i grundlæggende begreber fra programmering. Et vigtigt element er brugen af variabler, som bruges til at gemme data, f.eks. spillerens hastighed eller om en nøgle er blevet samlet op. Derudover er betingelser, altså if-statements, blevet brugt til at styre, hvad der sker i spillet, f.eks. når det afgøres, om spilleren dør, eller om spilleren kan komme igennem døren. Funktioner er også anvendt til at strukturere koden og gøre den mere overskuelig ved at samle bestemte handlinger et sted.

Et andet centralt begreb er kollisioner, som gør det muligt at registrere, når spilleren rammer objekter som platforme, fjender eller nøgler. Dette er en vigtig del af spillets egenskaber, da det blandt andet afgør, hvornår spilleren dør, eller hvornår en nøgle bliver samlet op. Der er også arbejdet med simple spilmekanikker som bevægelse og tyngdekraft, så spillet føles mere realistisk, når karakteren hopper eller falder.

Udviklingen af spillet er foregået i Godot, som er en game engine, der gør det muligt at arbejde med både grafik og kode i samme program. Her er spillet opbygget i forskellige scener, hvor hver del af spillet, f.eks. spiller, fjender og bane, er opdelt som separate elementer. Dette gør det lettere at holde overblik over projektet og arbejde med de enkelte dele af spillet.

I forhold til metode er spillet blevet udviklet ved hjælp af den iterative proces. Det betyder, at udviklingen foregår i flere trin, hvor produktet løbende forbedres gennem test og evaluering. I stedet for at udvikle hele spillet på én gang bliver funktionerne udviklet enkeltvist og derefter forbedret undervejs.

Der er også arbejdet med opdeling af projektet i mindre delopgaver. Denne metode gør det lettere at skabe overblik over projektet og fokusere på én funktion ad gangen. Ved at arbejde med enkelte systemer som bevægelse, kollisioner og levelskift separat bliver det lettere at teste og finde fejl i koden.

I præproduktionsfasen blev der arbejdet med kravspecifikation. En kravspecifikation bruges til at opstille de krav og funktioner, som produktet skal indeholde. Formålet med denne metode er at skabe en tydelig ramme for projektet og sikre, at de vigtigste funktioner bliver prioriteret først.

Der blev også anvendt brainstorming som metode i konceptudviklingen. Brainstorming bruges til at komme med idéer gennem fri idéudveksling uden at vurdere idéerne med det samme. Metoden anvendes ofte i kreative projekter

for at skabe mange forskellige forslag og muligheder, før der vælges en endelig idé.

Derudover blev der lavet skitser og prototyper. Skitser bruges til at visualisere, hvordan spillet skal se ud, f.eks. levels eller menuer. Prototyper er tidlige versioner af spillet, som bruges til at teste idéer og funktioner, før det endelige produkt bliver lavet.

Der blev også anvendt markedsundersøgelse som metode. En markedsundersøgelse bruges til at undersøge eksisterende produkter, målgrupper og tendenser inden for et bestemt område. Formålet er at få viden om lignende produkter og skabe et bedre udgangspunkt for udviklingen af ens eget produkt.

I forbindelse med målgruppeundersøgelsen blev der taget udgangspunkt i teorier om spillertyper. En målgruppeundersøgelse bruges til at analysere, hvilke brugere et produkt henvender sig til, og hvilke behov eller interesser de har. Dette gør det muligt at tilpasse design og funktioner til den ønskede målgruppe.

Der blev også udarbejdet en tidsplan som en del af projektstyringen. En tidsplan bruges til at strukturere arbejdsprocessen og skabe overblik over, hvornår forskellige opgaver skal udføres. Dette gør det lettere at planlægge projektforløbet og sikre, at arbejdet bliver gennemført inden for tidsrammen.

For at skabe struktur i arbejdsprocessen blev der arbejdet med rollefordeling. Rollefordeling bruges til at fordele ansvar og opgaver mellem gruppemedlemmerne, så arbejdet organiseres mere effektivt.

Under udviklingen blev der også arbejdet med trial and error som metode. Trial and error betyder, at man prøver forskellige løsninger af, indtil man finder en løsning, der virker. Denne metode anvendes ofte i programmering og problemløsning, da fejl og eksperimentering er en stor del af udviklingsprocessen.

Derudover blev kunstig intelligens (KI) anvendt som støtteværktøj under projektet. KI kan bruges til at hjælpe med at finde fejl i koden og komme med forslag til, hvordan koden i stedet skal være.

4 Analyse

Spillet er blevet lavet iterativt. De forskellige funktioner er blevet udviklet og testet løbende. Først blev de mest grundlæggende funktioner implementeret, som er at spilleren skal kunne bevæge sig rundt i banerne og have kollisioner med de forskellige instanser som banerne består af. Derfor blev der skrevet kode der tilføjede funktioner som bevægelse, animationer, og kollision, til karakteren. Efter dette blev der fremstillet andre instanser, som fjender, nøgler, udgange, killzones, osv. Med den iterative metode blev det muligt at opdage fejl løbende, i stedet for at hele spillet blev lavet også skulle det hele rettes til, til sidst, fordi der ville være en masse fejl der ikke var blevet opdaget. Funktionerne blev testet direkte i Godot, for at se om de virkede som forventet. Hvis de ikke gjorde dette, analyserede vi hvad problemet var og rettede det til, hvor efter der blev testet igen.

Spillet er opbygget ud fra scener i Godot. Den første scene der blev fremstillet, var spilleren, til spilleren blev der skrevet et script, som styrer de forskellige karakterer i de forskellige levels. Spillerens script styre bevægelser, hop, animationer og kollisioner. Bevægelsen fungerer ved at der kommer et input fra tastaturet, også bruges funktionen til at give en værdi til variabelen "direction":

```
1. var direction := Input.get_axis("move_left", "move_right")
```

Dette returnerer en værdi mellem -1 og 1 afhængigt af om spilleren går mod venstre eller højre. Udfra dette beregnes hastigheden:

```
1. const SPEED = 100.0  
2. velocity.x = direction * SPEED
```

SPEED er en konstant for bevægelseshastigheden i pixels per sekund. Så hvis hastigheden har en "direction" på -1 vil den bevæge sig mod venstre og hvis den er 1 så bevæger den sig mod højre. Når spilleren hopper, så bruges konstanten "JUMP_VELOCITY":

```
1. const JUMP_VELOCITY = -200.0  
2. if not is_on_floor():  
3.     velocity += get_gravity() * delta  
4. if Input.is_action_just_pressed("jump") and is_on_floor():  
5.     velocity.y = JUMP_VELOCITY
```

Hvis spilleren står på jorden og der bliver trykket på knappen for "jump" så vil hastigheden i y-retning blive sat til konstanten for "JUMP_VELOCITY"

hvilket vil resultere til at spilleren bevæger sig op. For at den også bevæger sig ned igen, er der også en "if" der tjekker om spilleren røre jorden. Hvis den ikke gør det, bliver der tilføjet tyngdekraft til spilleren som sørger for at den falder ned igen.

Hop, tyngdekraft og bevægelser er tilføjet med funktionen

```
1. func _physics_process(delta: float) -> void:
```

Denne funktion kører automatisk med et fast interval i forhold til FPS. Delta sørger for at spillet køre med samme hastighed på alle computere, da kraftigere computere har en højere FPS.

Fjenderne bruger en simpel kode, som er baseret på RayCasts. Fjenden bevæger sig hele tiden i en retning men når RayCast registrere en kollision med et objekt, så ændres "direction" så den går den modsatte vej, da princippet er det samme som spillerens hastighed.

```
1. func _process(delta):
2.     if ray_cast_right.is_colliding():
3.         direction = -1
4.         animated_sprite.flip_h = true
5.     if ray_cast_left.is_colliding():
6.         direction = 1
7.         animated_sprite.flip_h = false
8.     position.x += direction * SPEED * delta
```

Til point systemet er der lavet nøgler som skal bruges til at åbne døren for at man kan gennemføre banerne. Dette system er implementeret gennem en GameManager, der er gjort til et globalt script, dette gør at man kan henvise til det lige meget hvilken scene man skriver kode til. Derudover er der også en kode kun til nøglen. Når spilleren går ind i en nøgle registres det da der sker en kollision mellem spiller og nøglen. I GameManager er der lavet en variabel for "score" som starter med en værdi på 0, samt der er lavet en funktion til at tilføje point til scoren

```
1. var score = 0
2. func add_point():
3.     score += 1           # Øger scoren med 1
4.     print(score)
```

Denne globale funktion kan så blive kaldt i scriptet til nøglen.

Her bliver der brugt en funktion der hedder "_on_body_entered()". Denne funktion registrere om der er en body der går ind i nøglen:


```
1. func _on_body_entered(body: Node2D) -> void:
2.     if body.is_in_group("Player"):
3.         GameManager.add_point()
4.         animation_player.play("Pickup")
```

Når det bliver registreret, så bliver det tjekket om denne body tilhøre gruppen "player" for at den ikke tilføjer point hvis det er f.eks. en fjende der går ind i nøglen. Derefter bliver der tilføjet et point via GameManager med funktionen "GameManager.add_point()" og animationen der fjerner nøglen bliver spillet.

De point man får ved at samle en nøgle op bliver brugt til at bestemme om man kan åbne døren og slippe ud. Der bliver igen brugt samme funktion og testet om den body der er gået ind er en "player" så fjenden ikke ved et tilfælde får spillet til at skifte level.

Hvis disse krav er opfyldt, så bliver det tjekket om der er blevet samlet nøgler. Hvis der er ikke er det sker der ikke noget men hvis der er så bliver fil stien til level et hentet. Denne fil bliver lavet om til en Integer med "current_scene.file.to_int()". Det denne funktion gør er at fjerne alle ikke tal, så man får en talværdi ud. Dette virker da det eneste tal der er i navnet på filstierne, er nummeret på de levels der er. Så hvis man har filstien fra level 1: "res://scenes/levels/level_1.tscn", får man 1 ud af dette. Her lægges der 1 til for at få en værdi for næste level. Herefter opdateres GameManager så scoren nulstilles og så den også har styr på hvilket level man er på. Så ventes der en frame for at der ikke opstår fejl, da spilleren ellers vil stoppe midt i en bevægelse.

Nu kan det tjekkes om der er et level mere, da der i GameManager er defineret en max antal levels. Hvis der er et nyt level så sættes nummeret på det næste level ind som en string i filstien og man bliver ført til næste level. Hvis der derimod ikke er flere levels bliver man taget tilbage til spilmenuen.

```
1. if GameManager.score == 3:
2.     var current_scene_file = get_tree().current_scene.scene_file_path
3.     var next_level_number = current_scene_file.to_int() + 1
4.     GameManager.current_level += 1
5.     GameManager.score = 0
6.     await get_tree().process_frame
7.     if next_level_number <= GameManager.max_level:
8.         var next_level_path = "res://scenes/levels/level_" +
9.             str(next_level_number) + ".tscn"
10.        get_tree().change_scene_to_file(next_level_path)
```

```
10.     else:
11.         get_tree().change_scene_to_file("res://scenes/game.tscn")
```

5 Test og teststrategi

Programmet er blevet testet løbende gennem hele udviklingsprocessen som en del af den iterative metode. Hver gang der blev lavet en ny funktion, blev den testet direkte i Godot for at se, om den fungerede som forventet. På den måde kunne fejl findes og rettes tidligt i processen.

Der blev især testet spillerbevægelse, hop, tyngdekraft og kollisioner for at sikre, at spilleren kunne bevæge sig korrekt rundt i spillet. Derudover blev nøgle- og pointsystemet testet for at kontrollere, om nøgler blev samlet op korrekt, og om levelskift fungerede. Fjendernes bevægelse og killzones blev også testet for at sikre, at spilleren døde eller restartede banen korrekt.

Når der opstod fejl, blev koden ændret og testet igen. Denne metode kaldes trial and error og blev brugt flere gange under udviklingen. For at skabe overblik over testene er der lavet en tabel for testene med input, forventet output og faktisk output, som kan ses herunder:

Input	Forventet output	Faktisk output
Spilleren trykker på højre piletast / D	Karakteren bevæger sig mod højre	Karakteren bevægede sig mod højre som forventet
Spilleren trykker på venstre piletast / A	Karakteren bevæger sig mod venstre	Karakteren bevægede sig mod venstre som forventet
Spilleren trykker pil op / W	Karakteren hopper	Karakteren hoppede
Spilleren rører ikke jorden	Tyngdekraften får spilleren til at falde ned igen	Spilleren faldt ned som forventet
Spilleren rammer en platform	Spilleren stopper og falder ikke igennem platformen	Spilleren blev stående på platformen

Tabel 1: Test af spillets funktioner

Input	Forventet output	Faktisk output
Spilleren rammer en nøgle	Nøglen bliver samlet op, og scoren stiger med 1	Nøglen blev samlet op, og scoren steg korrekt
Spilleren går ind i døren uden nok nøgler	Spilleren kan ikke komme videre til næste level	Spilleren blev i samme level
Spilleren går ind i døren med nok nøgler	Spillet skifter til næste level	Levelsift fungerede korrekt
Spilleren rammer en fjende	Level genstarter	Level blev genstartet korrekt
Spilleren falder i en killzone	Spillet genstarter level	Level restartede korrekt
Fjenden rammer en væg	Fjenden vender retning	Fjenden vendte korrekt
Spilleren trykker på level-knap i menuen	Det valgte level åbnes	Det korrekte level blev åbnet
Spilleren gennemfører sidste level	Spilleren kommer tilbage til startmenuen	Spilleren kom tilbage til startmenuen
Spilleren holder hop-knappen nede	Spilleren skal ikke kunne hoppe mens den befinder sig i luften	Spilleren kunne ikke hoppe mens den var i luften
Spilleren står stille	Idle-animation afspilles	Idle-animationen fungerede korrekt
Spilleren løber	Run-animation afspilles	Run-animation fungerede korrekt
Spilleren hopper	Jump-animation afspilles	Jump-animation fungerede korrekt
Spillet startes	Startmenuen vises	Startmenuen blev vist korrekt

Tabel 2: Test af spillets funktioner

6 Konklusion

Formålet med projektet var at undersøge hvordan man kan designe og udvikle et 2D-spil i Godot med fokus på spillerinteraktion og spilmekanikker. Under udviklingen af spillet, blev det undersøgt, hvordan programmeringsprincipper som variabler, funktioner, if-statements, kollisioner og objektorienteret programmering kan anvendes til at fremstille en spil der virker.

Det kan konkluderes at Godot og GDScript er et godt værktøj til udvikling af simple 2D spil, da det ved hjælp af scener, scripts og indbyggede funktioner i Godot er lykkedes at skabe et spil med spilmekanikker som bevægelse, hop, tyngdekraft, kollisioner, fjender, opsamling af nøgler og level skift. Spillet reagerer også korrekt på de input der kommer fra tastaturet.

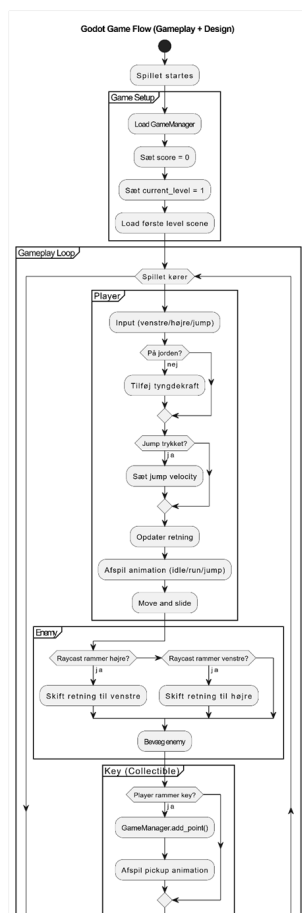
Arbejdet med dette projekt har vist at valget af programmering metode har stor betydning for det endelige produkt. Objektorienteret programmering har gjort det muligt at inddele projektet i klasser, som har gjort det let at sammensætte levels og arbejde iterativt ved at teste, finde fejl og rette dem. Samtidig gjorde det spillets struktur mere overskueligt at have separate scener og scripts.

Det kan ud fra testene vurderes at spillet virker helt som det skal og de forskellige funktioner virkede som forventet. Spillet er sammenhængende men der kunne dog godt laves flere levels i stedet for kun 2, så man får en længere spiloplevelse.

Samlet set kan det konkluderes at det har været muligt at udvikle en fungerende 2D-spil med de programmeringsmetoder og værktøjer der var til rådighed. Projektet har samtidig givet en bredere forståelse for spiludvikling, programmering og arbejdsprocesser i forbindelse med udviklingen af produktet.

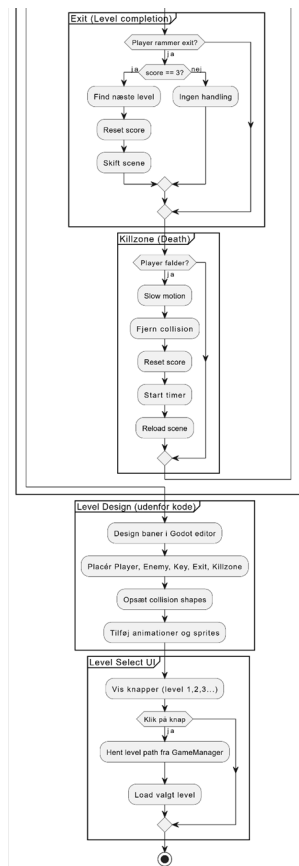
Bilag

Flowdiagram Del 1



Figur 1: Flowdiagram del 1

Flowdiagram Del 2



Figur 2: Flowdiagram del 2

Præproduktion

Kravspecifikation

- Vi skal holde os til de krav som der bliver stillet i opgavebeskrivelsen.
- Spillet skal:
 - Indeholde en spiller, der interagerer via tastatur eller mus
 - Indeholde dynamik i spillet (fx bevægelige objekter, skiftende spiltilstande, inputvalidering eller genererede løsninger)
 - Have mindst ét bevægeligt objekt
 - Indeholde point- eller livssystem
 - Have en start- og sluttilstand
- Andre krav:
 - Skal laves i Godot
 - 2D-spil
- Minimal viable product / need-to-have:
 - En spiller
 - En nøgle
 - En dør
 - En fare
 - Platforme
- Nice to have:
 - Tile-map som vi selv har designet/lavet

Konceptudvikling

- Ideudvikling:
 - Brainstorm:
 - * Flygt fra skole
 - Et spil hvor det handler om at samle nøgler så man kan slippe ud fra en låst skole
 - Det er et platformsspil

- Der er fjendtlige lærere som prøver at gøre så man ikke kan slippe ud fra skolen

- Spillets USP:

- Kombinationen af platformsspil og skoletema

Ressourcer

- Vi har programmet “Godot” til rådighed til at lave vores spil.
- Vi kan også bruge YouTube til at finde viden om, hvordan vi kan lave spillet.

Roller

- Projektleder – Lasse
- Programmør – Begge
- Spil designer – Begge
- Level designer – Sebastian
- Grafikker – Lasse
- Lyddesigner – Sebastian

Målgruppe

- Der skal udarbejdes en analyse af målgruppen så vi ved hvordan produktet skal designes og virke.
- Achievers kan godt lide at opnå ting. De bliver motiverede af at få point, komme til nye baner, samle udstyr, tjene penge og andre ting som kan måles og sammenlignes med andre spillere.
- Explorers kan godt lide at gå på opdagelse. De vil gerne finde nye områder og små hemmelige rum.
- Socializers synes at det sociale element er det vigtigste ved at spille.
- Killers søger altid kampen og bliver motiverede af at konkurrere mod andre spillere.

- Spillet er primært rettet mod achievers, da det handler om at samle nøgler og gennemføre baner.
- Spillet henvender sig også lidt til explorers, fordi spilleren skal udforske mappet for at finde alle nøglerne og hemmelige områder.

Konceptbeskrivelse

- Hvilken avatar kan man styre?
 - Man kan styre en elev der skal slippe ud fra skolen.
- Hvad er spillets regler?
 - Man skal slippe ud fra skolen.
 - For at kunne komme ud skal man samle nøgler så man kan åbne døren.
 - Man må ikke blive fanget af lærerne.
- Hvad er målet og hvordan vinder man?
 - Målet er at finde nøglerne og slippe ud fra skolen.
- Hvordan får man point?
 - Man bruger nøgler til at åbne døren.
 - Spilleren har også liv som mistes hvis man bliver fanget.
- Hvilke karakterer skal indgå i spillet?
 - Der er en lærer som fungerer som modstander.
- Hvordan skal historien udfolde sig?
 - Spilleren får at vide at man skal slippe ud fra skolen og skal derefter udforske området for at finde nøgler.
- Hvordan skal grafikken se ud?
 - Pixelgrafik.
- Hvordan skal lyden være?
 - Spooky lyde.
- Hvilken målgruppe er spillet rettet mod?
 - Achievers i alderen 12–16 år.

Markedsundersøgelse

- Vi har ikke kunnet finde lignende spil.
- Der findes spil hvor man skal flygte fra en skole, men de handler ofte om valg og klik frem for platform-gameplay.
- Kvantitativ analyse
- Kvalitativ analyse

SWOT-analyse

SWOT-analyse	
Stærke sider (Strengths)	Svage sider (Weaknesses)
Vi har valgt et simpelt spil, som vi realistisk kan nå at lave.	Vores spil kan komme til at virke meget simpelt.
Gameplayet er let at forstå for spilleren.	Vi er stadig nye til programmering, så vi kan få problemer med koden.
Vi kan bruge det vi har lært om variabler, if-statements og funktioner.	Vi har ikke meget erfaring med Godot endnu.
Spillet kan nemt udvides hvis vi bliver færdige med det basale.	Hvis vi ikke planlægger godt kan koden blive uoverskuelig.
Muligheder (Opportunities)	Trusler (Threats)
Vi kan gøre spillet mere interessant ved at tilføje flere funktioner.	Vi har begrænset tid i undervisningen.
Vi kan være kreative med design og banner.	Vi kan komme til at planlægge for mange features og ikke nå dem.
Vi kan målrette spillet til bestemte spillertyper.	Bugs og fejl kan tage lang tid at løse.
Vi kan bruge projektet til at blive bedre til programmering.	Projektet kan blive for stort i forhold til vores erfaring.

Tabel 3: SWOT-analyse

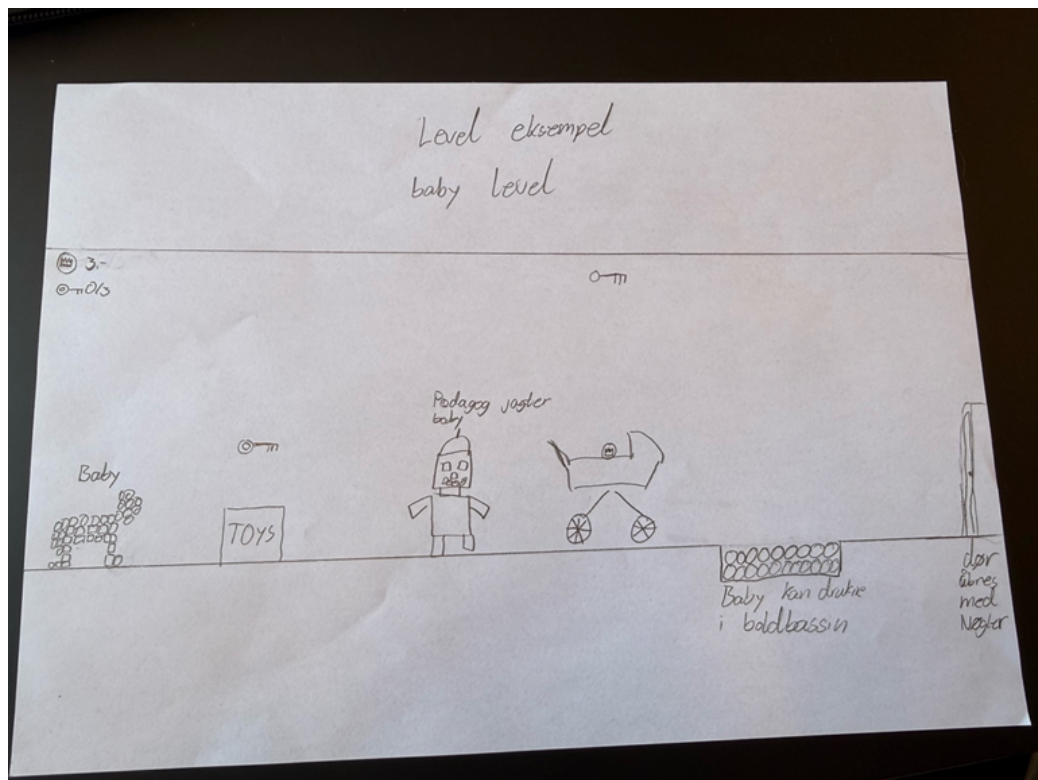
Tidsplan

Opgave	9	10	11	12	13	14	15	16	17	18	19
Præproduktion	P	P									
Lave sprites/tilemap			P								
Kode "Spilleren"			P	P	P						
Opbygge "Mappet"							P	P			
Lyd								P			
Løbende dokumentation		P	P	P	P		P	P	P	P	P
Løbende evaluering	P	P	P	P	P		P	P	P	P	P
Sammensætte spillet							P	P	P	P	P
Rapport							P	P	P	P	P

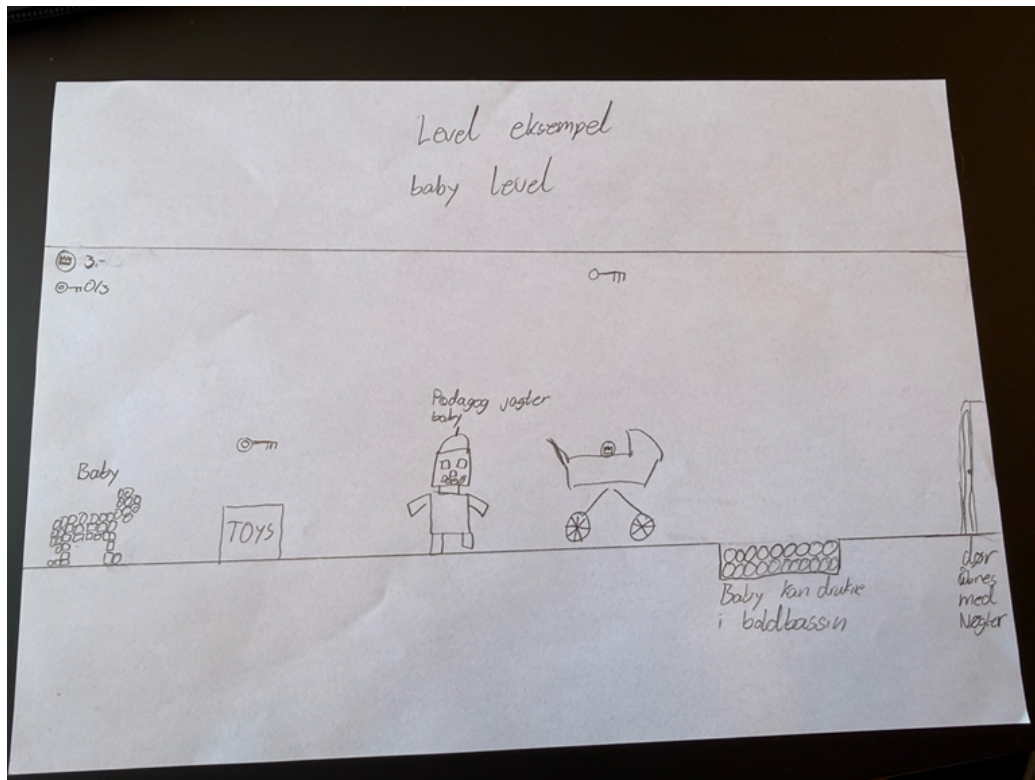
Figur 3: Tidsplan

Produktion

Skitser



Figur 4: Skitse af level 1



Figur 5: Skitse af spilmenu

Koordinering og prioritering

Meget vigtigt:

- Indeholde en spiller, der interagerer via tastatur eller mus
- Opsamling af nøgler til åbning af dør
- Platform
- Opsamling og optælling af nøgler
- Have en start- og sluttilstand
- Mindst 2 levels

Almindeligt:

- Fjende

- Zoner, hvor man kan “dø”

Mindre vigtigt:

- Eget tile-map
- Musik
- Mange levels